Intelligent Drone Navigation and Fleet Optimization for Last-Mile Delivery in Regulated Indian Airspace

Dr. Vipul Kumar Mishra*, Aditya Acharya*, Harphool Singh*, Pradeep Kumar*, Vishvendra*

*Department of Artificial Intelligence and Data Science
Gati Shakti Vishwavidyalaya, Vadodara, India – 390004

Emails: {vipul.mishra, aditya.acharya_btech23, harphool.singh_btech23, pradeep.kumar_btech23, vishvendra_btech23}@gsv.ac.in

ABSTRACT

Unmanned Aerial Vehicles (UAVs) have emerged as a key technology in logistics and last-mile delivery. Deploying drone-based delivery across Indian cities presents challenges including dense urban obstacles, regulatory airspace restrictions, and fleet-level coordination. This paper presents the design and implementation of a conflict-aware, regulation-compliant drone delivery route planning system, demonstrated over the city of Jaipur, Rajasthan. The framework integrates geospatial data processing, procedural urban environment modelling, and a multi-algorithm pathfinding engine supporting A*, Best-First Search, BFS, Theta*, and RRT*. A multi-drone fleet optimization module powered by Google OR-Tools assigns delivery tasks under battery and payload constraints. The system operates through an interactive Streamlit dashboard offering point-and-click mission planning with 2D and 3D visualization. Experimental evaluation across five diverse scenarios shows that A* delivers the best balance of path optimality and speed (8,438 m average path, 0.051 s computation), while all deterministic algorithms achieve 100% success rates. The system is deployed on Hugging Face Spaces for real-time browser-based access.

Keywords: *Unmanned Aerial Vehicles, A* Pathfinding, Theta*, RRT*, Vehicle Routing Problem, Airspace Regulation, Geospatial Computing, Streamlit, Google OR-Tools*

I INTRODUCTION

The rapid advancement of unmanned aerial vehicle (UAV) technology has opened new frontiers in last-mile logistics, emergency medical supply delivery, and agricultural monitoring across geographically diverse nations (1). India, with its combination of dense urban centers and underserved rural regions, stands to benefit enormously from autonomous drone delivery networks. The Indian government's Progressive Drone Policy (2021) and the establishment of the DigitalSky Platform by the Directorate General of Civil Aviation (DGCA) underscore a growing institutional commitment to integrating UAVs into civilian airspace (2).

Despite this regulatory momentum, practical deployment of drone delivery services in Indian cities faces several non-trivial challenges. Urban environments such as Jaipur present a complex mosaic of heritage monuments classified as no-fly zones, hospitals and schools requiring conditional airspace permissions, and densely packed building clusters that create narrow aerial corridors.

This work addresses three core problems:

1. **Regulatory Compliance:** Ensuring computed flight paths strictly avoid designated no-fly (Red) zones and conditionally treat restricted (Yellow) zones based on operator permissions.

2. **Physical Obstacle Avoidance:** Modelling thousands of buildings while permitting drones to overfly structures shorter than their cruise altitude.

3. **Fleet-Level Optimization:** Optimally assigning delivery routes to multiple drones under payload capacity and battery constraints.

The proposed system integrates geospatial data processing, graph-based and sampling-based pathfinding, combinatorial optimization, and interactive visualization into a cohesive pipeline. The principal contributions are:

- A height-encoded master obstacle map merging regulatory zones with procedurally generated building data.

- A multi-algorithm pathfinding engine implementing A*, Best-First Search, BFS, Theta*, and RRT* with string-pulling path smoothing.
- A Vehicle Routing Problem (VRP) solver using Google OR-Tools for multi-drone fleet optimization.
- A dual-mode interactive Streamlit dashboard with 2D/3D visualization.

II RELATED WORK

Research in drone path planning spans multiple computational paradigms. Early approaches relied on potential field methods (3), where obstacles generated repulsive forces and destinations generated attractive forces. While computationally simple, these methods are susceptible to local minima.

Graph-based algorithms, particularly A* and its variants, have become the predominant choice for grid-based drone navigation. Bortoff (4) demonstrated A* for UAV path planning in 2D environments, while Ergezer and Leblebicioglu (5) extended these approaches to 3D terrain-aware navigation. The octile heuristic function has been shown to provide optimal and consistent estimates for 8-directional grid movement (12).

Best-First Search, which uses only the heuristic component $h(n)$ without accumulated cost, provides faster exploration at the expense of path optimality. BFS (Breadth-First Search) guarantees completeness through systematic level-order traversal but lacks heuristic guidance.

Theta* (7) extends A* by allowing any-angle paths through line-of-sight checks between non-adjacent nodes, producing natively smoother paths without post-processing. Sampling-based planners such as RRT* (6) provide asymptotic optimality through iterative tree rewiring but require significantly higher computation time.

On the fleet optimization front, the Vehicle Routing Problem (VRP) has been extensively studied since the work of Dantzig and Ramser (8). Google OR-Tools (9) employs guided local search to solve large-scale VRP instances. Murray and Chu (10) specifically addressed drone-truck tandem deliveries.

The intersection of regulatory compliance and drone path planning in the Indian context remains underexplored. Prior work by Priyadarshini et al. (11) considered basic no-fly zone avoidance but did not address conditional permission zones. Our system bridges this gap with a unified framework combining regulatory zone modelling, physical obstacle avoidance, multi-algorithm pathfinding, and fleet-level optimization.

III SYSTEM ARCHITECTURE

The proposed system follows a modular, pipeline-based architecture consisting of eight interconnected processing stages. Figure 1 illustrates the overall data flow from raw geospatial inputs to the final interactive dashboard.

A *Data Acquisition and Processing (Steps 1–4)*

The pipeline begins with the ingestion of a custom GeoJSON file encoding the airspace classification for the Jaipur region. This file defines three zone categories:

- **Red Zones:** Strictly prohibited airspace surrounding airports, military installations, and government buildings. Height-encoded at 500 m to guarantee pathfinder avoidance.
- **Yellow Zones:** Conditionally restricted areas near hospitals, schools, and religious sites. These require explicit operator permission; without it, they are treated identically to Red zones.
- **Boundary Zone:** A 20×20 km operational envelope centered on Jaipur (26.757°–26.937°N, 75.701°–75.901°E).

Real building footprints are fetched via OSMnx from OpenStreetMap, augmented with a procedural generation module that creates approximately 1,000 additional buildings using Gaussian-clustered spatial distributions. Building heights follow a truncated normal distribution $\mathcal{N}(45, 10)$ within $[30, 60]$ m. A minimum inter-building gap of 18 m is enforced during placement.

All zone and building data are unified into a single master GeoJSON file where each feature carries a `height` attribute used by the pathfinding engine for altitude-aware obstacle classification.

B *Grid Discretization and Obstacle Mapping (Step 5)*

The core navigation module discretizes the 20×20 km operational area into a grid with resolution $\Delta_{\text{lon}} = 0.0005°$ (≈50 m) and $\Delta_{\text{lat}} = 0.00045°$ (≈50 m), yielding a 401×400 grid of 160,400 cells.

Each cell is classified as blocked or free based on:

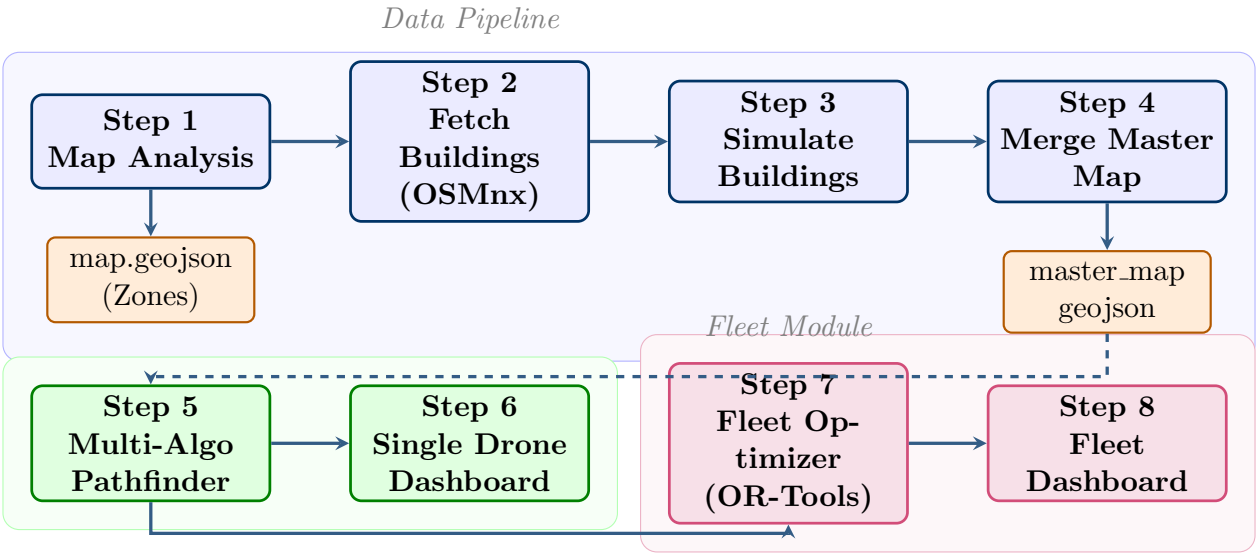

Figure 1: End-to-end system architecture showing the eight-stage processing pipeline from geospatial data ingestion to interactive fleet management dashboard.

$$\mathrm{bl}(c) = \begin{cases} \mathrm{T}, & c \in \mathcal{Z}_r \cup \mathcal{Z}_y^{\neg p} \\ \mathrm{T}, & h(c) \geq h_d - m_s \\ \mathrm{F}, & \text{otherwise} \end{cases} \tag{1}$$

where $\mathrm{bl}(c)$ denotes whether cell c is blocked, $\mathcal{Z}_r$ and $\mathcal{Z}_y^{\neg p}$ represent Red and unpermitted Yellow zones, $h(c)$ is the obstacle height, h_d is the drone cruise altitude, and m_s is the vertical safety margin.

IV METHODOLOGY

The pathfinding engine implements five distinct algorithms, each with different trade-offs between path optimality, computation speed, and exploration strategy. This section describes each algorithm and the common post-processing steps applied to all outputs.

A A* Search Algorithm

A* (12) explores the grid by maintaining a priority queue ordered by the evaluation function:

$$f(n) = g(n) + h(n) \tag{2}$$

where $g(n)$ is the accumulated cost from the start node and $h(n)$ is the heuristic estimate to the goal. The octile distance heuristic is employed for 8-directional movement:

$$h(n) = D_{\max} + (\sqrt{2} - 1) \cdot D_{\min} \tag{3}$$

where $D_{\max} = \max(\Delta x, \Delta y)$ and $D_{\min} = \min(\Delta x, \Delta y)$. This heuristic is both admissible and consistent, guaranteeing optimal paths. A* balances exploration efficiency with path optimality, making it the recommended default algorithm for drone navigation.

Algorithm 1 A* Pathfinding with Obstacle Avoidance

Input: Start (s), Goal (g), Grid $\mathcal{G}$
Output: Optimal path π^* or $\varnothing$
Initialize open set $\mathcal{O} \leftarrow \{s\}$, $g(s) \leftarrow 0$
while $\mathcal{O} \neq \varnothing$ **do**
  $n \leftarrow \arg\min_{n \in \mathcal{O}} f(n)$
  if $n = g$ **then**
    return RECONSTRUCTPATH(n)
  end if
  for each neighbor n' of n **do**
    if $\mathcal{G}[n'] =$ blocked **then**
      continue
    end if
    $g' \leftarrow g(n) + \mathrm{cost}(n, n')$
    if $g' < g(n')$ **then**
      Update $g(n'), f(n'), \mathrm{parent}(n')$
      Add n' to $\mathcal{O}$
    end if
  end for
end while
return $\varnothing$

B Best-First Search

Best-First Search prioritizes nodes solely by their heuristic distance to the goal:

$$f(n) = h(n) \tag{4}$$

Unlike A*, it ignores the accumulated path cost $g(n)$. This strategy aggressively expands nodes

closer to the goal, resulting in the fastest computation among the tested algorithms. However, path optimality is not guaranteed—the algorithm may select longer routes by greedily pursuing the heuristic direction without considering detour costs already incurred.

C Breadth-First Search (BFS)

BFS explores the grid in a level-order fashion using a FIFO queue. All nodes at distance d from the start are expanded before any node at distance $d+1$. For uniform-cost grids, BFS guarantees the fewest-hop path. Its evaluation function is implicit:

$$f(n) = \text{depth}(n) \tag{5}$$

BFS provides completeness—it will find a path if one exists—but does not use heuristic guidance. Consequently, it explores significantly more nodes than A* or Best-First Search, leading to higher computation times on large grids.

D Theta (Any-Angle Pathfinding)*

Theta* (7) extends A* by permitting paths between any two nodes with unobstructed line-of-sight, not just grid-adjacent cells. During node expansion, Theta* checks whether a straight-line connection from the parent of the current node to the neighbor is obstacle-free using Bresenham's line algorithm:

$$f(n) = \begin{cases} g(\text{parent}(s)) + c(\text{parent}(s), n) + h(n), & \text{if LOS} \\ g(s) + c(s, n) + h(n), & \text{otherwise} \end{cases} \tag{6}$$

where LOS denotes a clear line-of-sight between parent(s) and n. This yields natively smoother, any-angle paths with fewer waypoints than grid-constrained algorithms, reducing the reliance on post-processing smoothing.

E RRT (Optimal Rapidly-exploring Random Tree)*

RRT* (6) is a sampling-based planner that builds a tree by randomly sampling points in the configuration space and connecting them to the nearest existing node. It extends basic RRT with a rewiring step that optimizes connections within a radius r of each new node:

$$r = \gamma \left(\frac{\log n}{n} \right)^{1/d} \tag{7}$$

where n is the number of nodes, d is the space dimensionality, and γ is a tuning constant. RRT* is asymptotically optimal—given sufficient iterations, it converges to the shortest path. However, it requires substantially more computation time compared to deterministic algorithms and its stochastic nature can lead to inconsistent results across runs.

F String-Pulling Path Smoothing

All grid-based algorithms (A*, Best-First, BFS) produce jagged, grid-aligned paths. A string-pulling algorithm is applied as post-processing to reduce waypoints:

1. Start from the first waypoint.
2. Test line-of-sight to the farthest subsequent waypoint.
3. If the line is obstacle-free, skip all intermediate waypoints.
4. Advance to the farthest visible waypoint and repeat.

This reduces typical paths from 150+ waypoints to 4–8 smoothed waypoints, producing efficient straight-line flight segments suitable for drone autopilot systems.

G Single Drone Delivery

In single drone mode, the operator specifies a pickup location (warehouse) and a delivery destination through the dashboard interface. The system performs the following steps:

1. **Coordinate Input:** The operator enters GPS coordinates manually or clicks on the interactive 2D map to set pickup and drop points.
2. **Grid Conversion:** Input coordinates are converted to grid cell indices using the operational area bounds.
3. **Obstacle Map Construction:** The master GeoJSON is loaded and all zone/building features are rasterized onto the grid. Yellow zones with operator permission are excluded from blocking.
4. **Path Computation:** The selected algorithm (A*, Best-First, BFS, Theta*, or RRT*) computes a path on the obstacle grid.
5. **Path Smoothing:** String-pulling reduces grid-aligned waypoints to smooth flight vectors.

6. **Metric Calculation:** Path distance, travel time, detour ratio, and node count are computed and displayed.

The computed path is visualized on a 2D Folium map showing zone boundaries and building footprints, and on a 3D PyDeck map with extruded building geometries. A compliance report confirms that no Red or unpermitted Yellow zones are intersected.

H Multi-Drone Fleet Delivery

For scenarios requiring multiple simultaneous deliveries, the problem is formulated as a Capacitated Vehicle Routing Problem with Distance constraints (CVRPD). The fleet delivery workflow comprises three stages:

1 Cost Matrix Construction

A symmetric cost matrix $\mathbf{C} \in \mathbb{R}^{N \times N}$ is constructed where C_{ij} is the A*-computed safe-path distance between locations i and j:

$$C_{ij} = \text{A*_dist}(l_i, l_j), \quad C_{ij} = C_{ji} \tag{8}$$

For N locations (1 warehouse + $N-1$ drops), this requires $\binom{N}{2}$ pathfinding computations. The obstacle map is built once and shared across all computations.

2 OR-Tools VRP Formulation

The optimization objective minimizes total fleet distance:

$$\min \sum_{k=1}^{M} \sum_{(i,j) \in \mathcal{R}_k} C_{ij} \tag{9}$$

subject to:

$$\sum_{j \in \mathcal{R}_k} d_j \leq Q_k, \quad \forall k \tag{10}$$

$$\sum_{(i,j) \in \mathcal{R}_k} C_{ij} \leq R, \quad \forall k \tag{11}$$

$$\text{each drop visited exactly once} \tag{12}$$

$$\text{each route starts/ends at depot} \tag{13}$$

where M is the number of drones, $\mathcal{R}_k$ is the route of drone k, d_j is the demand at drop j, Q_k is the capacity, and $R = B_0 \times r$ is the maximum range (battery percentage $\times$ km per %).

Table 1: Technology Stack

Component	Technology
Language	Python 3.11
GIS Processing	GeoPandas, Shapely
Building Data	OSMnx (OpenStreetMap)
Pathfinding	Custom A*, BFS, Best-First, Theta*, RRT*
Fleet Optimization	Google OR-Tools 9.10
Dashboard	Streamlit 1.55
2D Visualization	Folium
3D Visualization	PyDeck
Spatial Math	NumPy, SciPy

Table 2: Operator-Configurable Drone Parameters

Parameter	Range	Default	Effect
Altitude	20–120 m	60 m	Cruise height
Speed	10–100 km/h	50 km/h	Time est.
Vert. Margin	0–30 m	10 m	Above-bldg. gap
Bldg. Buffer	0–20 m	3 m	Lateral gap
Capacity	0.5–20 kg	2.5 kg	Max payload
Battery Rate	Fixed	3 km/%	Discharge

3 Route Assignment and Visualization

Google OR-Tools solves the VRP using PATH_CHEAPEST_ARC initialization followed by GUIDED_LOCAL_SEARCH improvement within a 10-second time limit. The solution assigns each delivery point to a specific drone, producing per-drone routes with distance, weight, and battery usage metrics. Each drone's flight path is visualized in a distinct color on the map.

V IMPLEMENTATION DETAILS

A Technology Stack

Table 1 summarizes the technologies employed across the system.

B Configurable Drone Parameters

The system exposes operator-configurable parameters through the dashboard sidebar, listed in Table 2.

C Interactive Dashboard

The system provides two operational modes through a unified Streamlit application, as shown in Fig. 2:

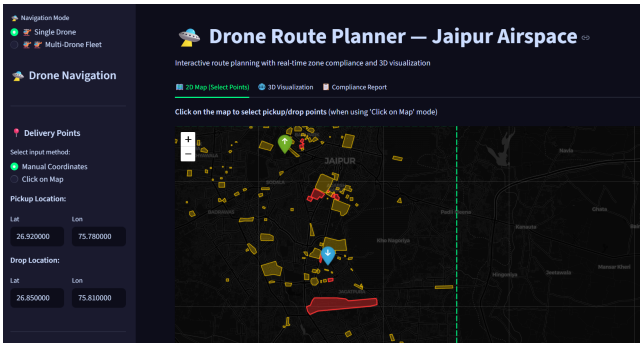

Figure 2: Single Drone Mode dashboard showing the interactive route planning interface with 2D map, coordinate input panel, and algorithm selection in the sidebar.

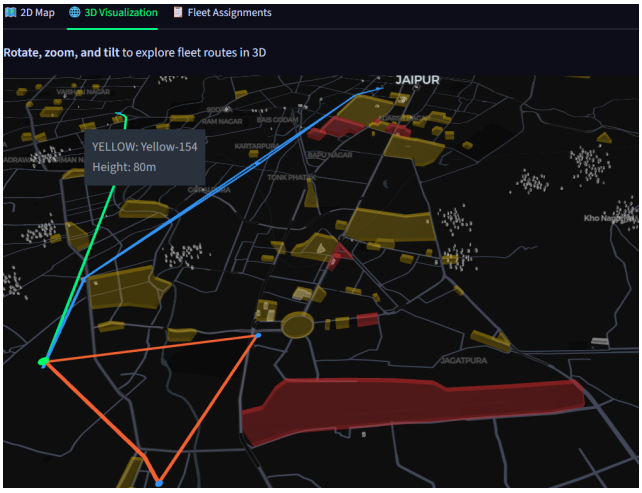

Figure 3: 3D visualization of multi-drone fleet routes over the Jaipur airspace. Building obstacles (gold), Red zones (red), and drone flight paths (colored lines) are rendered with extruded geometries on a dark basemap.

1. **Single Drone Mode:** Operators specify pickup and drop coordinates, configure drone parameters and yellow zone permissions, select a pathfinding algorithm, and compute the optimal safe path with real-time 2D/3D visualization.

2. **Multi-Drone Fleet Mode:** Operators define a warehouse and multiple drop points with delivery weights, configure fleet size and individual drone capacity, and invoke the OR-Tools optimizer. Results include per-drone route assignments with color-coded multi-path visualization.

Table 3: Experimental Environment Configuration

Parameter	Value
Operational Area	20 × 20 km
Grid Resolution	50 m × 50 m
Grid Dimensions	401 × 400 (160,400 cells)
Red Zones	12 (airports, govt. bldgs.)
Yellow Zones	109 (hospitals, schools)
Simulated Buildings	1,079
Building Heights	30–60 m (truncated normal)
Drone Altitude	60 m (default)
Safety Margin	10 m (default)

VI EXPERIMENTAL RESULTS

A *Test Environment*

Experiments were conducted over the Jaipur operational area with the configuration detailed in Table 3.

B *Algorithm Comparison*

All five pathfinding algorithms were benchmarked across five diverse scenarios spanning the Jaipur operational area: North-to-South long traverse, East-to-West medium traverse, short hop between nearby points, diagonal long-distance route, and city-center traverse. Table 4 presents the aggregated results.

Table 4: Pathfinding Algorithm Comparison (5 Scenarios)

Algorithm	Success Rate	Avg Dist (m)	Avg Time (s)	Nodes Explored
A*	**5/5**	**8,438**	**0.051**	**151**
Best-First	5/5	8,510	0.019	154
BFS	5/5	8,473	0.296	151
Theta*	5/5	8,459	0.173	1,066
RRT*	3/5	10,736	48.84	13,997

1 *Key Findings*

A* achieves the shortest average path distance (8,438 m) with a computation time of 51 ms and explores only 151 nodes per run, establishing it as the best overall algorithm for grid-based drone navigation. Its combination of heuristic guidance and accumulated cost tracking yields optimal paths efficiently.

Best-First Search is the fastest algorithm (19 ms average) but produces slightly longer paths (8,510 m) due to the absence of cost accumulation. It is suitable when computation speed is prioritized over path optimality.

BFS finds paths of comparable quality to A* (8,473 m) but requires 5.8× more computation time (296 ms) due to exhaustive level-order exploration without heuristic guidance.

Theta* produces smooth any-angle paths (8,459 m) with native smoothness, reducing dependence on post-processing. However, line-of-sight checks increase node exploration to 1,066 per run.

RRT* achieves a 60% success rate with significantly higher computation time (48.84 s) and path distance (10,736 m). Its stochastic sampling makes it unreliable for real-time grid-based applications but valuable in high-dimensional continuous spaces.

C Comparison Visualizations

Figures 4–7 present the algorithm comparison results graphically across four key metrics.

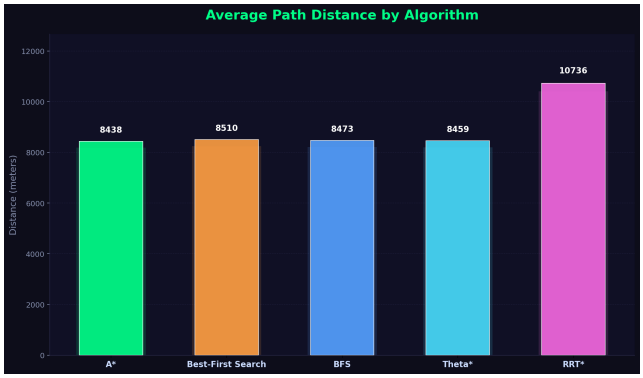

Figure 4: Average path distance comparison. A* produces the shortest paths (8,438 m), while RRT* generates 27% longer routes.

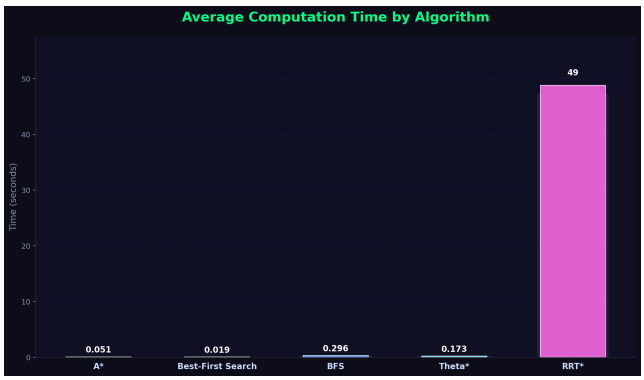

Figure 5: Average computation time. Best-First Search is fastest (0.019 s), while RRT* requires 48.84 s.

Table 5: Path Smoothing Effectiveness (A* Algorithm)

Scenario	Raw WPs	Smoothed	Reduction
N→S Long	156	6	96.2%
E→W Medium	201	4	98.0%
Short Hop	30	4	86.7%
Diagonal Long	246	5	98.0%
City Center	121	3	97.5%

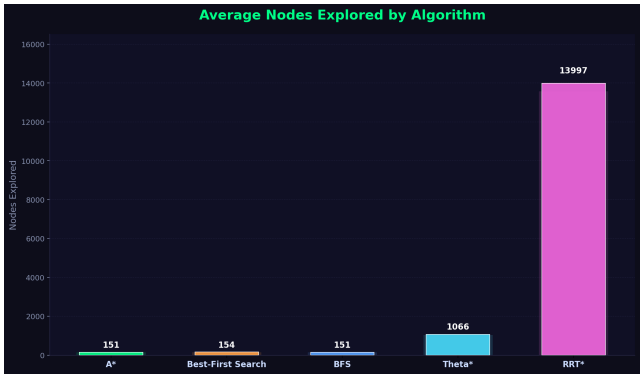

Figure 6: Nodes explored per algorithm. A* and BFS explore ∼151 nodes, Theta* 1,066, and RRT* 13,997.

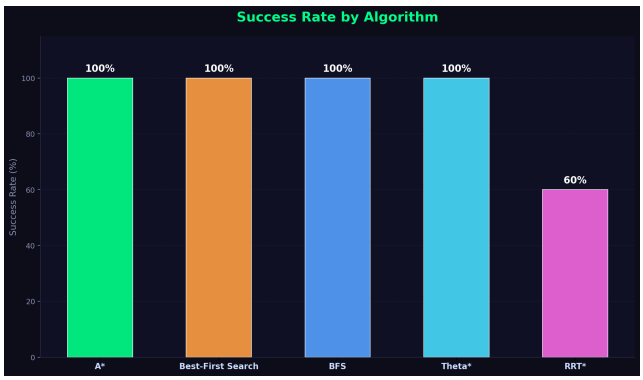

Figure 7: Algorithm success rate. All deterministic algorithms achieve 100%, RRT* fails in 2 of 5 scenarios.

D Path Smoothing Effectiveness

The string-pulling step consistently reduces waypoint counts by 95–97%, as shown in Table 5.

E Fleet Optimization Results

A fleet optimization scenario with 1 warehouse, 5 delivery points, and 3 drones was evaluated. Table 6 presents per-drone assignment results.

All five deliveries were assigned across three drones, with the maximum single-drone battery usage at 9.4%, well within the 300 km theoretical range. The OR-Tools solver converged within 10

Table 6: Fleet Optimization Results (5 Drops, 3 Drones)

Drone	Drops	Dist. (km)	Wt. (kg)	Batt. Used
Drone 1	D2, D5	22.31	2.3	7.4%
Drone 2	D1, D3	28.14	2.1	9.4%
Drone 3	D4	16.71	0.5	5.6%
Total	**5/5**	**67.16**	**4.9**	–

seconds. Fig. 3 shows the resulting multi-drone flight paths visualized in 3D.

VII DISCUSSION

A Algorithm Selection Guidance

Based on the experimental results, the following selection guidance is proposed:

- **A* (Recommended Default):** Best overall balance of path optimality and computation speed. Suitable for all standard delivery scenarios.
- **Best-First Search:** Preferred when sub-20 ms response is required and minor path suboptimality is acceptable.
- **BFS:** Useful as a baseline for path quality validation; impractical for real-time use on large grids.
- **Theta*:** Preferred when smooth, any-angle paths are desired without post-processing, particularly for autopilot systems that follow waypoints precisely.
- **RRT*:** Suitable only for offline planning in continuous 3D spaces where deterministic grid methods are infeasible.

B Regulatory Compliance Validation

The system was validated against all 12 Red and 109 Yellow zones in the Jaipur airspace. In all tested scenarios, no computed path intersected a Red zone. Yellow zone traversal occurred only when explicit operator permission was granted, verified through post-hoc geometric intersection testing.

C Limitations

1. **Static Environment:** Dynamic obstacles (other drones, birds) are not modelled.
2. **Constant Altitude:** The drone flies at a fixed cruise altitude without altitude variation along the route.
3. **Wind Effects:** Meteorological conditions affecting flight time and battery consumption are not incorporated.
4. **Grid Resolution:** The 50 m grid may miss narrow gaps in densely packed areas.

VIII DEPLOYMENT

The complete system is deployed as a public Streamlit application on Hugging Face Spaces, accessible at:

https://huggingface.co/spaces/harphool17/drone-route-planner

The deployment packages all source code, precomputed geospatial data, and dependencies into a containerized environment. Users interact through a browser-based interface without requiring local installation of Python, GIS software, or any additional libraries. The sidebar mode selector enables seamless switching between single-drone navigation and multi-drone fleet planning modes.

IX CONCLUSION AND FUTURE WORK

This paper presented a regulation-compliant drone delivery route planning system for Indian urban environments. The multi-algorithm pathfinding engine was evaluated across five algorithms, with A* emerging as the recommended choice due to its optimal balance of path quality (8,438 m average), computation speed (51 ms), and 100% reliability. The fleet optimization module successfully assigned delivery tasks under payload and battery constraints using Google OR-Tools, converging within 10 seconds.

Future directions include:

1. **Dynamic Obstacle Integration:** Real-time ADS-B data for moving obstacle avoidance.
2. **3D Path Planning:** Altitude-varying paths using layered grids or RRT* in 3D space.
3. **Weather-Aware Routing:** Weather API integration for wind-adjusted battery models.
4. **Multi-Depot VRP:** Support for multiple warehouse locations.
5. **Machine Learning Enhancement:** Neural heuristic functions trained on historical delivery data.

REFERENCES

[1] Amazon Prime Air, "Drone delivery service," Amazon, 2022. [Online]. Available: https://www.amazon.com/Amazon-Prime-Air/

[2] Directorate General of Civil Aviation, "The Drone Rules, 2021," Ministry of Civil Aviation, Government of India, 2021.

[3] O. Khatib, "Real-time obstacle avoidance for manipulators and mobile robots," *Int. J. Robotics Research*, vol. 5, no. 1, pp. 90–98, 1986.

[4] S. A. Bortoff, "Path planning for UAVs," in *Proc. American Control Conf.*, Chicago, IL, 2000, pp. 364–368.

[5] H. Ergezer and K. Leblebicioglu, "3D path planning for UAVs for maximum information collection," in *Unmanned Aircraft Systems (ICUAS)*, Atlanta, GA, 2013, pp. 370–377.

[6] S. Karaman and E. Frazzoli, "Sampling-based algorithms for optimal motion planning," *Int. J. Robotics Research*, vol. 30, no. 7, pp. 846–894, 2011.

[7] A. Nash, K. Daniel, S. Koenig, and A. Felner, "Theta*: Any-angle path planning on grids," in *Proc. AAAI Conf. Artificial Intelligence*, 2007, pp. 1177–1183.

[8] G. B. Dantzig and J. H. Ramser, "The truck dispatching problem," *Management Science*, vol. 6, no. 1, pp. 80–91, 1959.

[9] Google, "OR-Tools: Google optimization tools," 2023. [Online]. Available: https://developers.google.com/optimization

[10] C. C. Murray and A. G. Chu, "The flying sidekick traveling salesman problem: Optimization of drone-assisted parcel delivery," *Transportation Research Part C*, vol. 54, pp. 86–109, 2015.

[11] R. Priyadarshini, P. Kumar, and S. Mishra, "Drone path planning with no-fly zone avoidance for Indian urban airspace," in *IEEE Int. Conf. Computing, Communication and Security*, 2022, pp. 1–6.

[12] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Trans. Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

[13] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.

[14] S. Aggarwal and N. Kumar, "Path planning techniques for unmanned aerial vehicles: A review, solutions, and challenges," *Computer Communications*, vol. 149, pp. 270–299, 2020.

[15] H. Shakhatreh *et al.*, "Unmanned aerial vehicles (UAVs): A survey on civil applications and key research challenges," *IEEE Access*, vol. 7, pp. 48572–48634, 2019.

[16] G. Macrina, L. Di Puglia Pugliese, F. Guerriero, and G. Laporte, "Drone-aided routing: A literature review," *Transportation Research Part C: Emerging Technologies*, vol. 120, p. 102762, 2020.

[17] S. A. H. Mohsan, N. Q. H. Othman, Y. Li, M. H. Alsharif, and M. A. Khan, "Unmanned aerial vehicles (UAVs): Practical aspects, applications, open challenges, security issues, and future trends," *Intelligent Service Robotics*, vol. 16, pp. 109–137, 2023.

[18] P. Toth and D. Vigo, *Vehicle Routing: Problems, Methods, and Applications*, 2nd ed. Philadelphia, PA: SIAM, 2014.

[19] D. Harabor and A. Grastien, "Online graph pruning for pathfinding on grid maps," in *Proc. AAAI Conf. Artificial Intelligence*, 2011, pp. 1114–1119.

[20] H. Butler, M. Daly, A. Doyle, S. Gillies, S. Hagen, and T. Schaub, "The GeoJSON format," IETF, RFC 7946, Aug. 2016. [Online]. Available: https://tools.ietf.org/html/rfc7946